Inteligência Artificial
Perceptron de várias camadas

Prof. Dr. Jefferson Beethoven Martins

IFTM Campus UPT – 09/03/2026

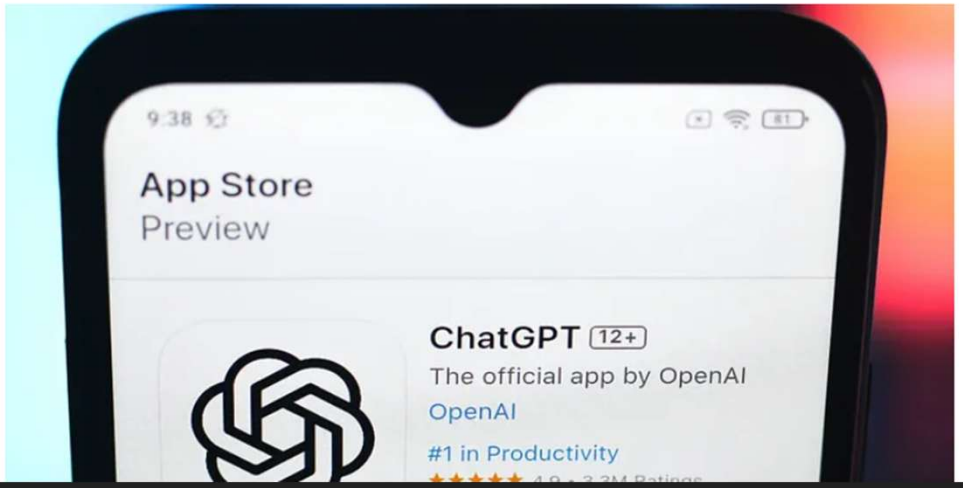
ChatGPT 5.4 já está entre nós — e aqui estão as tarefas em que ele se destacou | Exame

exame.

Home > Inteligência Artificial

ChatGPT 5.4 já está entre nós — e aqui estão as tarefas em que ele se destacou

Novo modelo da OpenAI aposta em raciocínio mais profundo, análise de documentos longos e produção de materiais de trabalho como planilhas e apresentações.



Microsoft diz que serviços da Anthropic continuarão disponíveis após EUA 'banir' empresa | Exame

exame.

Home > Inteligência Artificial

Microsoft diz que serviços da Anthropic continuarão disponíveis após EUA 'banir' empresa

Pedido dos EUA para evitar usar serviços da Anthropic não afetará consumidores comuns da Microsoft, diz empresa em nota

Maria Eduarda Cury
Colaboradora

Publicado em 6 de março de 2026 às 10h15.

A Microsoft disse que os produtos da **Anthropic** permanecerão disponíveis para os clientes da empresa mesmo após o banimento definido pelo **Pentágono**. O governo do presidente **Donald Trump** requisitou que as agências federais deixassem de utilizar as ferramentas da companhia responsável pelo Claude Code em decorrência da recusa da Anthropic de aceitar o acordo com o Departamento de Guerra dos **Estados Unidos**, mas a Microsoft diz que seus produtos permanecerão inalterados.

CEO da Anthropic diz que funcionários permanecem na empresa por 'missão', e não salários | Exame

exame.

Home > Inteligência Artificial

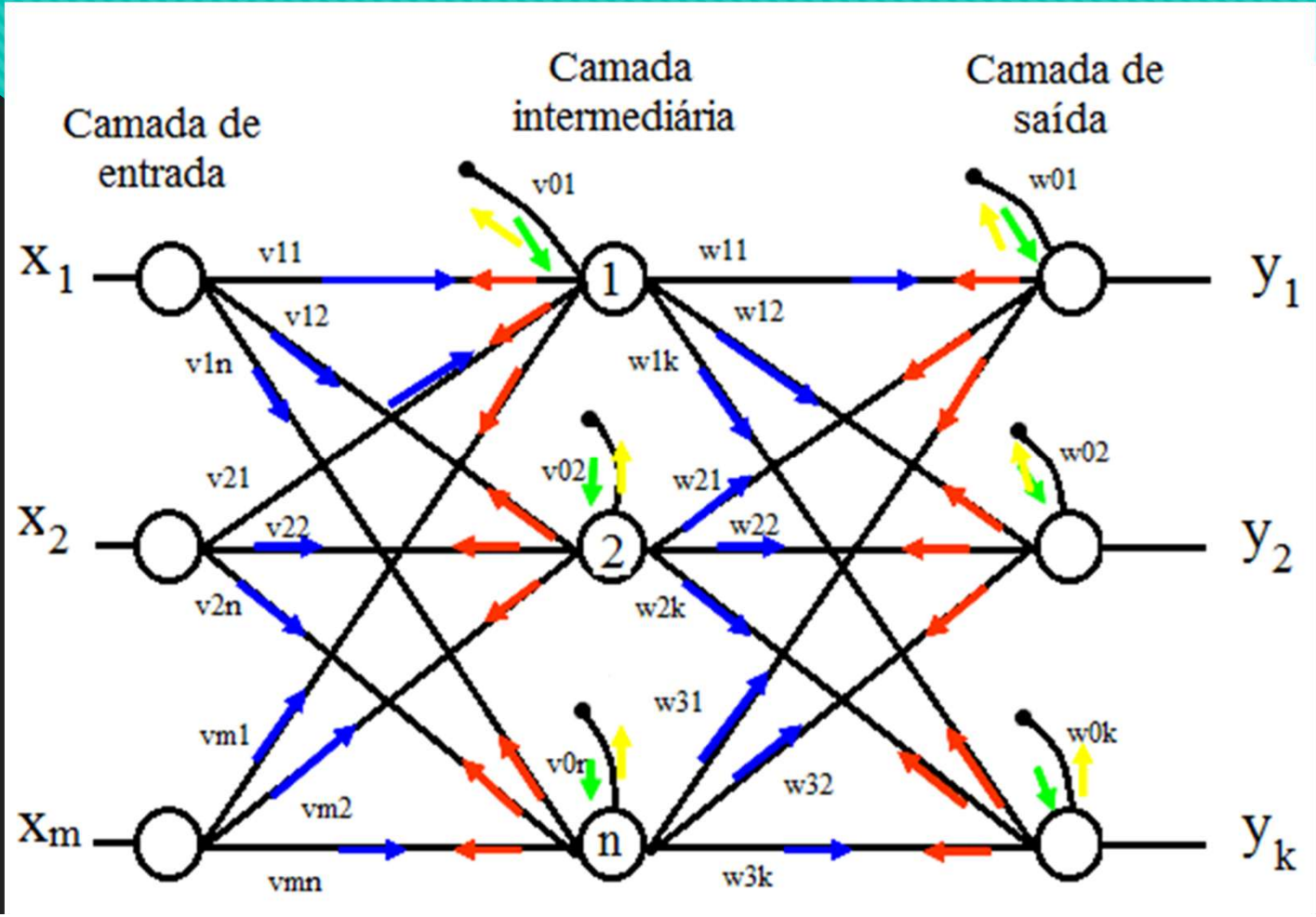
CEO da Anthropic diz que funcionários permanecem na empresa por 'missão', e não salários

Dario Amodei disse em reunião com investidores que trabalhadores recusam salários milionários da Meta por cultura diferenciada da Anthropic

Redes Multilayer Perceptron - descrição

- Possuem pelo o menos uma camada de neurônios além da camada de entrada e a camada de saída.
- Essa(s) camada(s) adicionais são denominadas como camadas intermediárias ou camadas escondidas (*hidden layers*).
- São extremamente versáteis quanto a possibilidade de aplicações: reconhecimento de padrões, aproximação de funções, previsão de séries temporais, identificação e controle de processos e otimização de sistemas.
- É considerada como pertencente à arquitetura *feedforward* de camadas múltiplas, cujo treinamento é do tipo *supervisionado*.
- Não há qualquer realimentação de neurônios da camada de saída na camada de entrada.

Redes Multilayer Perceptron - arquitetura



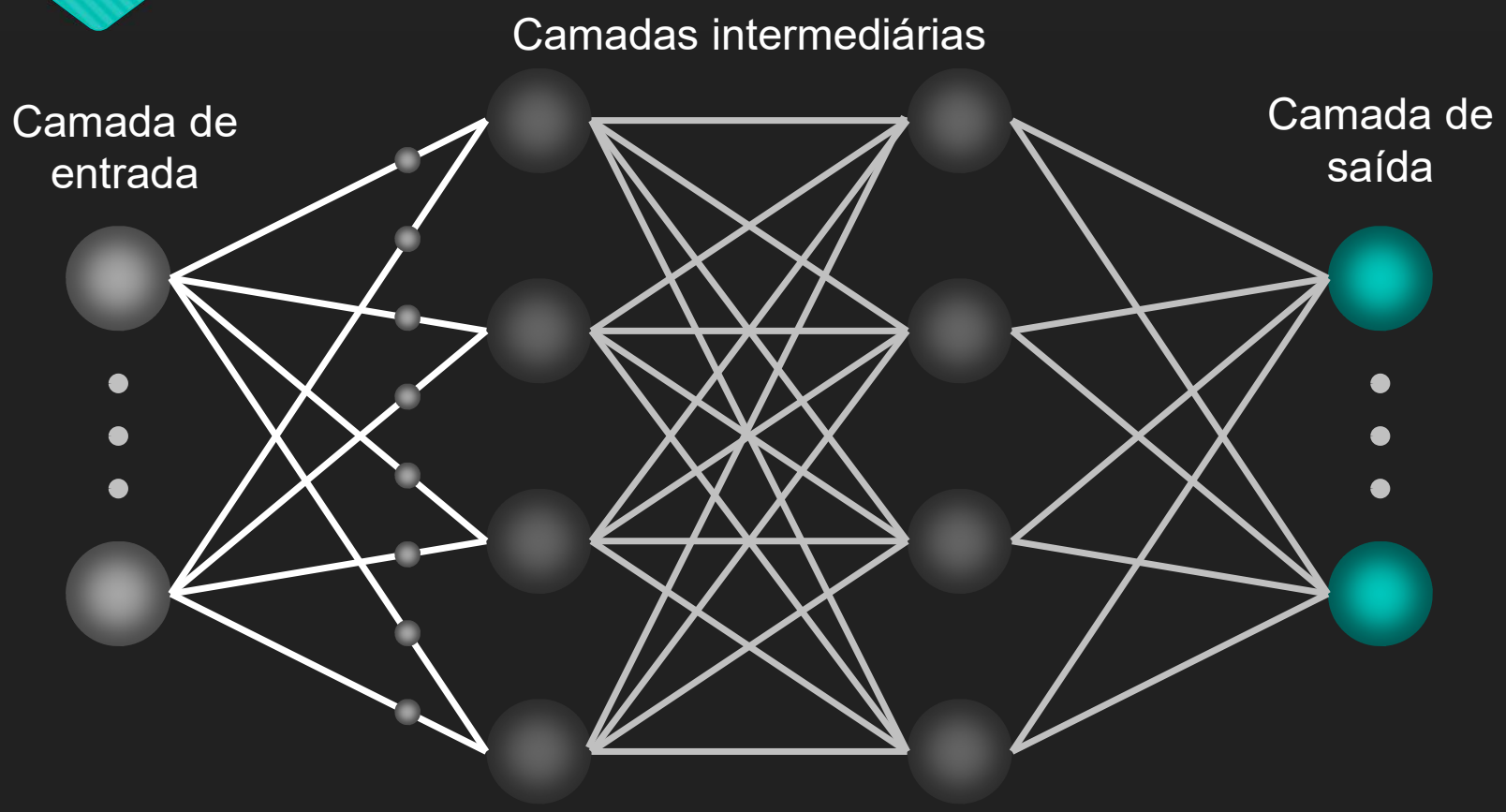
Redes Multilayer Perceptron – História e treinamento

- O início de sua popularidade e vasta aplicação em diversos problemas, se deu em 1986 com a Publicação do livro *Paralell Distributed Processing* de autoria de Geoffrey Hinton, David Rumelhart e Ronald J. Williams.
- Nessa publicação os autores apresentam o algoritmo de treinamento *Backpropagation* (Retropropagação do erro), que permite o treinamento de redes com pelo o menos uma camada intermediária.
- O algoritmo *backpropagation* utiliza a regra delta generalizada para atualização dos pesos, assim como as redes Adaline.
- Diferente do perceptron básico, as redes Multilayer perceptron podem ter mais de uma saída.
- O problema da "porta XOR" foi resolvido (não linearmente separável).

Redes Multilayer Perceptron - funcionamento

- A entrada da rede recebe os sinais advindos do problema.
- Estes sinais são processados pelos pesos sinápticos e pelas camadas intermediárias é feita a extração da maioria das informações referentes ao comportamento dos dados.
- Finalmente os neurônios da camada de saída recebem os estímulos provenientes da última camada intermediária, produzindo a resposta disponibilizada pela rede.
- A arquitetura da MLP: quantidade de entradas, quantidade de camadas intermediárias, quantidade de neurônios em cada camada intermediária, número de neurônios da camada de saída; depende de diversas variáveis.

Redes Multilayer Perceptron Propagação

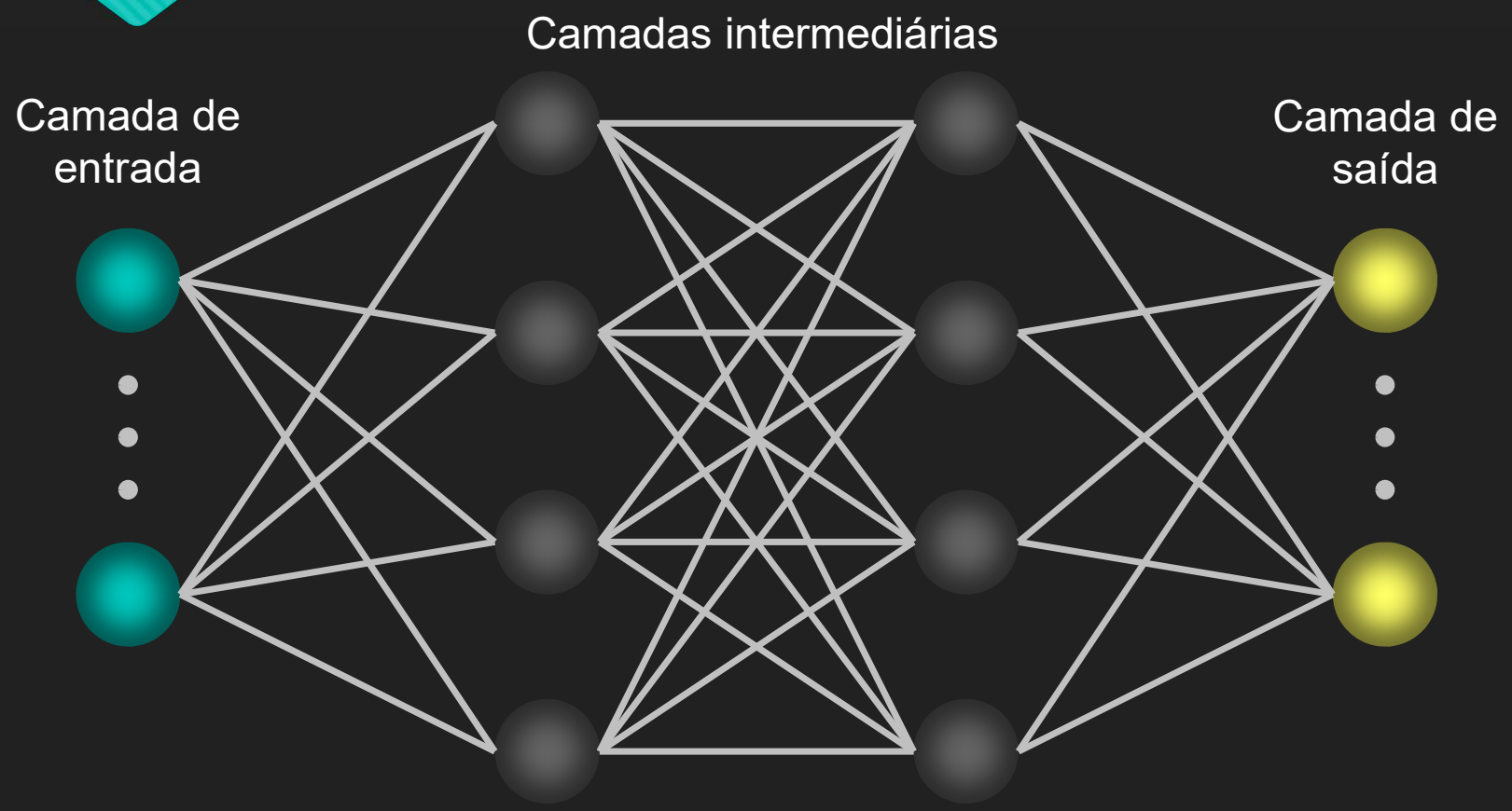


Redes Multilayer Perceptron Propagação

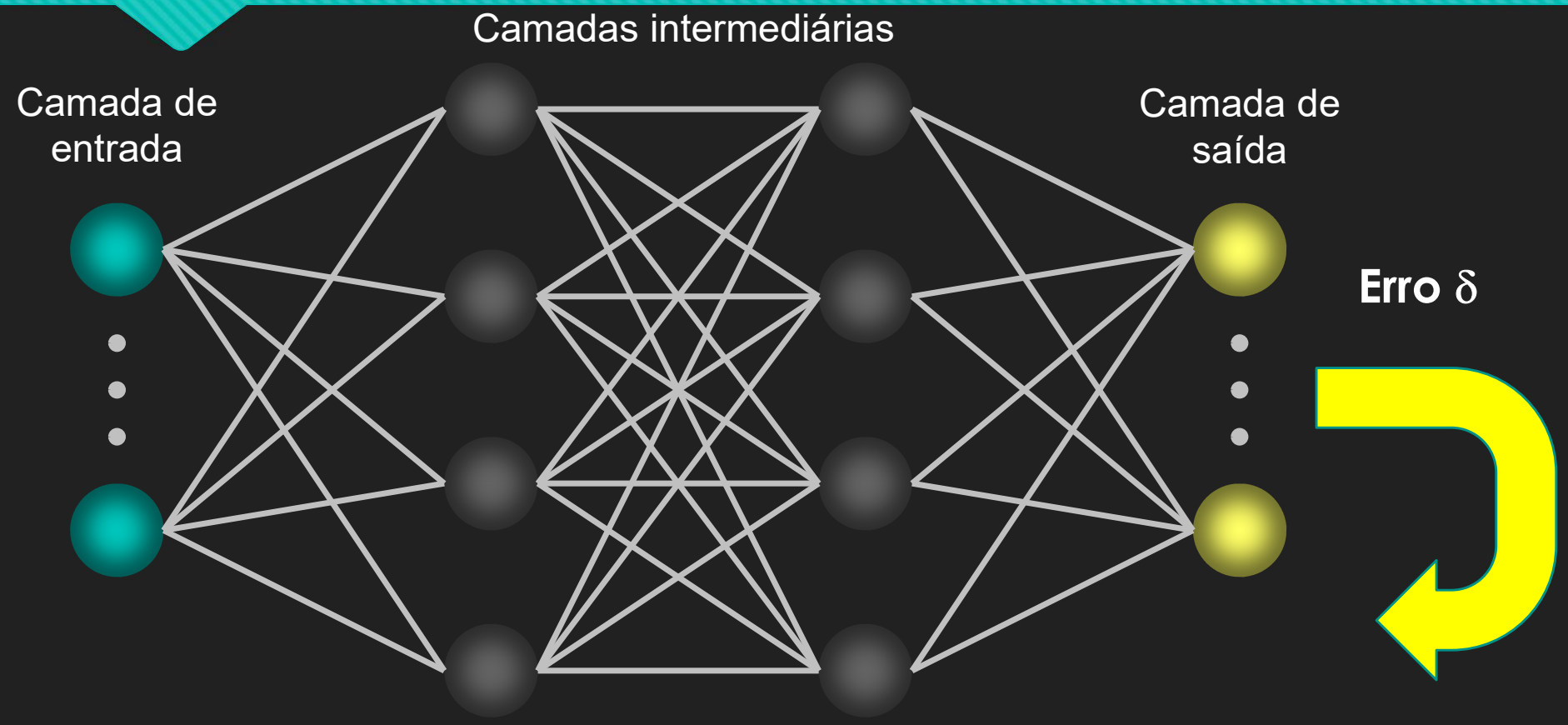
Redes Multilayer Perceptron Propagação

Redes Multilayer Perceptron Propagação

Redes Multilayer Perceptron Propagação



Redes Multilayer Perceptron Propagação



Redes Multilayer Perceptron
Algoritmo backpropagation

- Regra de aprendizagem baseada na correção do erro pelo método do Gradiente descendente.
- O algoritmo de aprendizagem é composto de duas fases:

a) Cálculo do erro (*forward*)

b) Correção dos pesos sinápticos (*backward*)

Redes Multilayer Perceptron
Algoritmo backpropagation

INSTITUTO FEDERAL
Triângulo Mineiro

Valores que chegam a camada intermediária

Bias da camada intermediária

$$zin_j = v_{0j} + \sum_i x_i v_{ij}$$

Pesos entre a camada de entrada e a camada intermediária

Valores de entrada

Função de ativação

Valores que saem da camada intermediária

$$z_j = f\,(zin_j)$$

Valores que chegam a camada de saída

$$yin_k = w_{0k} + \sum_j z_j w_{jk}$$

Pesos entre a camada de entrada e a camada intermediária

Bias da camada de saída

Função de ativação

$$E = \frac{1}{2} \sum_k [t_k - y_k]^2$$

Valores que saem da camada de saída

$$y_k = f\,(yin_k)$$

Erro

Redes Multilayer Perceptron
Algoritmo backpropagation

Passo 1 – Inicialize os pesos (valores aleatórios pequenos)

Passo 2 – Enquanto a condição de parada não for satisfeita, execute as instruções de 1 a 6

Instrução 1 – Cada unidade de entrada (x_i, $i = 1, 2, ..., m$) recebe um sinal de entrada x_i e propaga este sinal para todas as unidade da camada a frente (as unidades escondidas).

Instrução 2 – Cada unidade escondida (z_j, $j = 1, 2, ..., n$) realiza a soma dos sinais de entrada ponderadas pelos pesos:

$$zin_j = v_{0j} + \sum_i x_i v_{ij}$$

o resultado anterior é submetido à função de ativação,

$$z_j = f(zin_j)$$

Redes Multilayer Perceptron
Algoritmo backpropagation

Este resultado é enviado para as unidades da camada seguinte (ou para a camada de saída caso exista apenas uma camada intermediária).

Instrução 3 – Cada unidade de saída (y_i, $i = 1, 2, ..., k$) realiza a soma de todos os sinais advindos da camada anterior ponderados pelos pesos sinápticos,

$$yin_k = w_{0k} + \sum_j z_j w_{jk}$$

e aplica-se a função de ativação.

$$y_k = f\,(yin_k)$$

Redes Multilayer Perceptron
Algoritmo backpropagation

Retropropagação do erro

Instrução 4 – Cada unidade de saída recebe um alvo (target) correspondente aquele padrão de treinamento, que é utilizado para calcular o erro.

$$E = \frac{1}{2}\sum_{k}[t_k - y_k]^2$$

de posse do erro, é obtida uma expressão que será utilizada para a correção dos pesos sinápticos.

$$\delta_k = [t_k - y_k]' f'(y_in_k)$$

Redes Multilayer Perceptron
Algoritmo backpropagation

A expressão para atualização dos pesos entre a camada intermediária e a camada de saída, e os pesos dos bias, são dadas por:

$$\Delta w_{jk} = \alpha \delta_k z_j,$$

$$\Delta w_{0k} = \alpha \delta_k,$$

O resultado de δk é utilizado para correção dos pesos da camada anterior.

Instrução 5 – Cada unidade escondida (z_j, j = 1, 2, ..., n) computa os deltas de entrada a partir da camada a frente.

$$\delta_in_j = \sum_{k=1}^{m} \delta_k w_{jk}$$

Redes Multilayer Perceptron
Algoritmo backpropagation

Em seguida, a derivada da função de ativação faz com que o termo de ajuste dos pesos entre a camada de entrada e a camada intermediária seja dado por:

$$\delta_j = \delta_in_j f'(z_in_j),$$

Essa última expressão é usada para as fórmulas de atualização dos pesos.

$$\Delta v_{ij} = \alpha \delta_j x_i,$$

$$\Delta v_{0j} = \alpha \delta_j,$$

Redes Multilayer Perceptron
Algoritmo backpropagation

Atualização de pesos e bias

Instrução 6 – Os pesos e bias utilizados no próximo ciclo de treinamento são obtidos pelas fórmulas abaixo:

$$w_{jk}\,(\text{new}) = w_{jk}\,(\text{old}) + \Delta w_{jk}.$$

$$v_{ij}\,(\text{new}) = v_{ij}\,(\text{old}) + \Delta v_{ij}.$$

Passo 3 – Testar a condição de parada.

Redes Multilayer Perceptron
Algoritmo backpropagation

Existem dois critérios clássicos para concluir o treinamento da MLP:

1) Alcance de um erro quadrático total máximo estabelecido antes do início do treinamento.
2) Alcance de um número de ciclos estabelecido antes do início do treinamento.

Redes Multilayer Perceptron – Fase de operação

Após a conclusão do treinamento, para obter a saída referente a uma amostra qualquer, basta realizar as seguintes operações.

$$zin_j = [x] \cdot [v_{ij}] + [v0_j]$$

$$z_j = [f(zin_1) \quad f(zin_2)]$$

$$yin_k = [z] \cdot [w_{jk}] + [w0]$$

$$y_k = [f(yin_1) \quad f(yin_2) \quad f(yin_3)]$$

A saída obtida deve ser comparada com as saídas atribuídas a cada padrão, e por meio dessa comparação será possível atribuir a qual classe aquele padrão pertence.

Redes Multilayer Perceptron – Fase de operação

Em problemas de reconhecimento de padrões a identificação da classe que compreende o padrão submetido é feita por meio da "limiarização" de cada componente da saída da rede e comparação com os alvos correspondentes a cada padrão. Por exemplo:

$$y = \begin{bmatrix} -0,7 \\ -0,2 \\ -0,8 \\ 0,4 \\ -0,3 \\ -0,1 \end{bmatrix} \rightarrow y = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$

a	b	c	d	e	f
1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1
-1	-1	1	-1	-1	-1
-1	-1	-1	1	-1	-1
-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	1

Padrão d

Exemplo: Treinar a rede MLP para diferenciar os três padrões abaixo.

Padrão	x_1	x_2	x_3
1	1	0,5	-1
2	0	0,5	1
3	1	-0,5	-1

Para fins de ilustração, vamos utilizar a taxa de aprendizagem igual a 0,01; os pesos sinápticos serão gerados aleatoriamente com valores entre -0,5 e 0,5; iremos trabalhar com uma camada intermediária com 2 neurônios e utilizaremos a função de ativação *tanh*().

Exemplo: Estabelecendo os vetores-alvo (targets)

Saída	Padrão 1	Padrão 2	Padrão 3
t_1	1	-1	-1
t_2	-1	1	-1
t_3	-1	-1	1

Exemplo: Apresentação da primeira amostra de treinamento.

$$zin_j = [x] \cdot v_{ij} + v0_j$$

$$zin_j = \left[\sum_{i=1}^{3} x_i \cdot v_{i1} \quad \sum_{i=1}^{3} x_i \cdot v_{i2} \right] + [v0_1 \quad v0_2]$$

$$zin_j = \underbrace{[1 \quad 0,5 \quad -1]}_{\text{Entradas da amostra 1}} \cdot \underbrace{\begin{bmatrix} 0,12 & -0,03 \\ -0,04 & 0,15 \\ 0,31 & -0,41 \end{bmatrix}}_{\text{Pesos } v_{ij} \text{ gerados aleatoriamente}} + \underbrace{[-0,09 \quad 0,18]}_{\text{Pesos (bias) } v0_j \text{ gerados aleatoriamente}}$$

$$zin_j = [1 \cdot 0,12 + 0,5 \cdot (-0,04) + (-1) \cdot 0,31 \quad 1 \cdot (-0,03) + 0,5 \cdot 0,15 + (-1) \cdot (-0,41)]$$
$$+ [-0,09 \quad 0,18]$$

$$zin_j = [-0,21 \quad 0,455] + [-0,09 \quad 0,18] = [-0,3 \quad 0,63]$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$z_j = \left[f\left(zin_1\right) \quad f\left(zin_2\right) \right]$$

$$z_j = f\left(zin_j\right)$$

$$z_j = \left[\tanh\left(-0,3\right) \quad \tanh\left(0,63\right) \right]$$

$$z_j = \left[-0,29 \quad 0,56 \right]$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$yin_k = [z] \cdot w_{jk} + w0_k$$

$$yin_k = \left[\sum_{i=1}^{3} z_j \cdot w_{j1} \quad \sum_{i=1}^{3} z_j \cdot w_{j2} \quad \sum_{i=1}^{3} z_j \cdot w_{j3} \right] + [w0_1 \quad w0_2 \quad w0_3]$$

$$yin_k = \underbrace{[-0,29 \quad 0,56]}_{\text{Valores obtidos em } z_j} \cdot \underbrace{\begin{bmatrix} -0,05 & -0,34 & 0,21 \\ 0,19 & -0,09 & 0,26 \end{bmatrix}}_{\text{Pesos } w_{jk} \text{ gerados aleatoriamente}} + \underbrace{[0,18 \quad -0,27 \quad -0,12]}_{\text{Pesos (bias) } w0_k \text{ gerados aleatoriamente}}$$

$$yin_k = \begin{bmatrix} (-0,29)\cdot(-0,05) & (-0,29)\cdot(-0,34) & (-0,29)\cdot 0,21 \\ +0,56\cdot 0,19 & +0,56\cdot(-0,09) & +0,56\cdot 0,26 \end{bmatrix}$$

$$+[0,18 \quad -0,27 \quad -0,12]$$

$$yin_k = [0,12 \quad 0,05 \quad 0,08] + [0,18 \quad -0,27 \quad -0,12] = [0,3 \quad -0,22 \quad -0,04]$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$y_k = \left[f(yin_1) \quad f(yin_2) \quad f(yin_3) \right]$$

$$y_k = f(yin_k)$$

$$y_k = \left[\tanh(0,3) \quad \tanh(-0,22) \quad \tanh(-0,04) \right]$$

$$y_k = \left[0,29 \quad -0,22 \quad -0,04 \right]$$

$$errototal = errototal + \frac{1}{2} \sum_{i=1}^{k} \left((t_i - y_i)^2 \right)$$

$$errototal = 0 + \frac{1}{2} \left((1 - 0,29)^2 + (-1 - (-0,22))^2 + (-1 - (-0,04))^2 \right)$$

$$errototal = \frac{1}{2} (0,5041 + 0,6084 + 0,9216) = 1,01705$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$\delta_k = (t_k - y_k)\cdot f'(yin_k) = (t_k - y_k)\cdot*(1+y_k)\cdot*(1-y_k)$$

$$\delta_k = \left(\begin{bmatrix}1\\-1\\-1\end{bmatrix} - \begin{bmatrix}0,29\\-0,22\\-0,04\end{bmatrix}\right)\cdot*\begin{bmatrix}1+0,29\\1+(-0,22)\\1+(-0,04)\end{bmatrix}\cdot*\begin{bmatrix}1-0,29\\1-(-0,22)\\1-(-0,04)\end{bmatrix}$$

$$\delta_k = \begin{bmatrix}0,71\\-0,78\\-0,96\end{bmatrix}\cdot*\begin{bmatrix}1,29\\0,78\\0,96\end{bmatrix}\cdot*\begin{bmatrix}0,71\\1,22\\1,04\end{bmatrix} = \begin{bmatrix}0,71\cdot 1,29\cdot 0,71\\(-0,78)\cdot(0,78)\cdot 1,22\\(-0,96)\cdot 0,96\cdot 1,04\end{bmatrix} = \begin{bmatrix}0,65\\-0,74\\-0,96\end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$\Delta w_{jk} = \alpha \cdot \delta_k \cdot z_j$$

$$\Delta w_{jk} = 0,01 \cdot \begin{bmatrix} 0,65 \\ -0,74 \\ -0,96 \end{bmatrix} \cdot \begin{bmatrix} -0,29 & 0,56 \end{bmatrix}$$

$$\Delta w_{jk} = 0,01 \cdot \begin{bmatrix} 0,65 \cdot (-0,29) & 0,65 \cdot 0,56 \\ (-0,74) \cdot (-0,29) & (-0,74) \cdot 0,56 \\ (-0,96) \cdot (-0,29) & (-0,96) \cdot 0,56 \end{bmatrix}$$

$$\Delta w_{jk} = 0,01 \cdot \begin{bmatrix} -0,19 & 0,36 \\ 0,21 & -0,41 \\ 0,28 & -0,54 \end{bmatrix} = \begin{bmatrix} -0,0019 & 0,0036 \\ 0,0021 & -0,0041 \\ 0,0028 & -0,0054 \end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$\Delta w0_k = \alpha \cdot \delta_k$$

$$\Delta w0_k = 0,01 \cdot \begin{bmatrix} 0,65 \\ -0,74 \\ -0,96 \end{bmatrix} = \begin{bmatrix} 0,0065 \\ -0,0074 \\ -0,0096 \end{bmatrix}$$

$$\delta in_j = \sum_{k=1}^{m} \delta_k w_{jk} = [\delta_k]^T \cdot [w_{jk}]^T$$

$$\delta in_j = \begin{bmatrix} 0,65 & -0,74 & -0,96 \end{bmatrix} \cdot \begin{bmatrix} -0,05 & 0,19 \\ -0,34 & -0,09 \\ 0,21 & 0,26 \end{bmatrix}$$

$$\delta in_j = \begin{bmatrix} -0,02 & -0,06 \end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$\delta_j = \delta in_j \cdot f'\left(zin_j\right)$$

$$\delta_j = \delta in_j \cdot *\left(1+z_j\right)\cdot *\left(1-z_j\right)$$

$$\delta_j = \begin{bmatrix}-0,02 & -0,06\end{bmatrix}\cdot *\left(\begin{bmatrix}1 & 1\end{bmatrix}+\begin{bmatrix}-0,29 & 0,56\end{bmatrix}\right)\cdot *\left(\begin{bmatrix}1 & 1\end{bmatrix}-\begin{bmatrix}-0,29 & 0,56\end{bmatrix}\right)$$

$$\delta_j = \begin{bmatrix}-0,02 & -0,06\end{bmatrix}\cdot *\begin{bmatrix}0,71 & 1,56\end{bmatrix}\cdot *\begin{bmatrix}1,29 & 0,44\end{bmatrix}$$

$$\delta_j = \begin{bmatrix}(-0,02)\cdot 0,71\cdot 1,29 & (-0,06)\cdot 1,56\cdot 0,44\end{bmatrix} = \begin{bmatrix}-0,02 & -0,04\end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$\Delta v_{ij} = \alpha \delta_j x_i$$

$$\Delta v_{ij} = \alpha \cdot \left[\delta_j\right]^T \cdot x_i$$

$$\Delta v_{ij} = 0,01 \cdot \begin{bmatrix} -0,02 \\ -0,04 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0,5 & -1 \end{bmatrix}$$

$$\Delta v_{ij} = 0,01 \cdot \begin{bmatrix} 0,4 & 0,2 & -0,4 \\ 0,32 & 0,16 & -0,32 \end{bmatrix} = \begin{bmatrix} -0,020 & -0,010 & 0,020 \\ -0,040 & -0,020 & 0,040 \end{bmatrix}$$

$$\Delta v0_j = \alpha \delta_j$$

$$\Delta v0_j = 0,01 \cdot \begin{bmatrix} -0,02 & -0,04 \end{bmatrix} = \begin{bmatrix} -0,0002 & -0,0004 \end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$w_{jk}(novo) = w_{jk} + \Delta w_{jk} = \left[w_{jk} \right] + \left[\Delta w_{jk} \right]^T$$

$$w_{jk}(novo) = \begin{bmatrix} -0,05 & -0,34 & 0,21 \\ 0,19 & -0,09 & 0,26 \end{bmatrix} + \begin{bmatrix} -0,0019 & 0,0021 & 0,0028 \\ 0,0036 & -0,0041 & -0,0054 \end{bmatrix}$$

$$w_{jk}(novo) = \begin{bmatrix} -0,0519 & -0,3379 & 0,2128 \\ 0,1936 & -0,0941 & 0,2546 \end{bmatrix}$$

$$w0_k(novo) = w0_k + \Delta w0_k = \left[w0_k \right] + \left[\Delta w0_k \right]^T$$

$$w0_k(novo) = \begin{bmatrix} 0,18 & -0,27 & -0,12 \end{bmatrix} + \begin{bmatrix} 0,0065 & -0,0074 & -0,0096 \end{bmatrix}$$

$$w0_k(novo) = \begin{bmatrix} 0,1865 & -0,2774 & -0,1296 \end{bmatrix}$$

Exemplo: Apresentação da primeira amostra de treinamento.

$$v_{ij}(novo) = v_{ij} + \Delta v_{ij} = \left[v_{ij}\right] + \left[\Delta v_{ij}\right]^T$$

$$v_{ij}(novo) = \begin{bmatrix} 0,12 & -0,03 \\ -0,04 & 0,15 \\ 0,31 & -0,41 \end{bmatrix} + \begin{bmatrix} -0,020 & -0,040 \\ -0,010 & -0,020 \\ 0,020 & 0,040 \end{bmatrix}$$

$$v_{ij}(novo) = \begin{bmatrix} 0,100 & -0,070 \\ -0,050 & 0,130 \\ 0,330 & -0,370 \end{bmatrix}$$

$$v0_j(novo) = v0_j + \Delta v0_j = \left[v0_j\right] + \left[\Delta v0_j\right]$$

$$v0_j(novo) = \begin{bmatrix} -0,09 & 0,18 \end{bmatrix} + \begin{bmatrix} -0,0002 & -0,0004 \end{bmatrix}$$

$$v0_j(novo) = \begin{bmatrix} -0,0902 & 0,1796 \end{bmatrix}$$

Exemplo: Preparação para apresentação da 2ª amostra de treinamento

$$v_{ij} = v_{ij}(novo)$$

$$v0_j = v0_j(novo)$$

$$w_{jk} = w_{jk}(novo)$$

$$w0_{jk} = w0_{jk}(novo)$$

Com os novos pesos, a segunda amostra de treinamento é apresentada, considerando o seu alvo correspondente, no caso:

$$\begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}$$

Exemplo: Continuando o treinamento.

- A apresentação da segunda amostra, bem como da terceira, segue os mesmos procedimentos de cálculo feitos para a apresentação da primeira amostra.

- O erro quadrático total é cumulativo, ou seja, o erro obtido com a apresentação da primeira amostra, deve ser adicionado com o obtido na segunda, com o obtido com a terceira, e com o obtido até a apresentação da enésima amostra.

- Após a apresentação de todas as amostras, completa-se um ciclo e testa-se a condição de parada. Se a condição de parada não for atendida, o erro quadrático total é reiniciado com valor zero e todas as amostras são reapresentadas novamente até completar o novo ciclo.

Exemplo: Continuando o treinamento.

- Os pesos não são reiniciados! O novo ciclo começa exatamente com os pesos atualizados com a apresentação da última amostra do ciclo anterior.

- O treinamento prossegue até que a condição de parada seja satisfeita.

Redes Multilayer Perceptron – Tamanhos das matrizes de treinamento

Considerando que a rede tenha uma única camada intermediária, ***m*** entradas, ***n*** neurônios na camada intermediária e ***k*** saídas, as matrizes de treinamento terão as seguintes dimensões:

$$v_{ij} = \left[v_{ij} \right]_{m \times n}$$

$$v0_j = \left[v0_j \right]_{1 \times n}$$

$$w_{jk} = \left[w_{jk} \right]_{n \times k}$$

$$w0_k = \left[w0_k \right]_{1 \times k}$$

$$\delta_k = \left[\delta_k \right]_{k \times 1}$$

$$\Delta w = \left[\Delta w \right]_{k \times n}$$

$$\Delta w0 = \left[\Delta w0 \right]_{k \times 1}$$

$$\delta in = \left[\delta in \right]_{1 \times n}$$

$$\delta_j = \left[\delta_j \right]_{1 \times n}$$

$$\Delta v = \left[\Delta v \right]_{n \times m}$$

$$\Delta v0 = \left[\Delta v0 \right]_{1 \times n}$$

Tarefinha

- Faça um pequeno programa para executar o exemplo apresentado (Uncle Jeff não admitirá Python).